

PRESALES PROPOSAL · AI AGENT · CODE ANALYSIS

售前提案 AI Agent

系统架构 · 处理逻辑 · 实现方法 · 优化路线

C# 14 · .NET 10 · ASP.NET Core

Anthropic 官方 SDK 直调 Messages API (claude-opus-4-8)

Dapper + MySQL 8 · 原生零依赖前端

把「售前方案编写」变成一条可重复执行的流水线

一名售前工程师原本需要 2-3 天完成的「读国标 → 选模块 → 配平台 → 组工作站 → 写方案 → 出 WORD」，本系统用一次 Claude 驱动的自动化流程在数十分钟内产出可交付初稿，且全部技术参数强制溯源到本地知识库。

97

个模块卡片

references/01-modules
模块 ID 即文件名

2

阶段生成

HTML / URS →
详细设计方案 Markdown

3

个受控工具

list / read / write
唯一的文件出入口

11

项流程国标

potocol/MD
前处理流程的唯一事实源

四项核心能力

1

模块选定

结构化输出 + 知识库校验，从 97 个模块中
选出最小充分集

2

平台选定

按平台约束（≤6 模组 / ≤8 托盘）匹配平台
抽象类

3

工作站组装

模组挂载到平台，按前处理工序串联；平台节
拍对齐 600 秒

4

解决方案书写

解决方案 HTML + 模块 URS + 详细设计方
案 + docx

设计红线：参数必须来自 references/ 与 potocol/，严禁编造；价格一律回复「请联系销售团队获取报价」；模型只能通过 3 个受控工具触碰磁盘。

全栈 .NET，零前端构建链，机密不入仓

后端运行时	C# 14 / .NET 10 / ASP.NET Core Web API	Controllers + DI + BackgroundService，单进程自托管
大模型	Anthropic 官方 NuGet 包 12.34.1 · claude-opus-4-8	进程内 HTTPS 直调 Messages API，Effort=high，自适应思考
数据访问	Dapper 2.1.79 + MySqlConnection 2.6.1	全参数化查询防注入；多值字段以 JSON 列存储
数据库	MySQL 8.x · SmartLabOS-Presales-AI	2 张表：PresalesProject / PresalesGeneration，建表脚本幂等
文档导出	DocumentFormat.OpenXml 3.5.1 + Pandoc(可选)	Markdown → docx 双路径，自动择优、离线可用
前端	原生 HTML / CSS / JS (零依赖，与 API 同源)	无需 npm/构建；静态文件禁缓存，避免加载旧脚本

机密治理：appsettings.json 仅存占位符 → 由 dotnet user-secrets 或环境变量 ANTHROPIC_API_KEY / ConnectionStrings_SmartLabOS 覆盖，仓库零机密。

五层单体：请求即返回，重活全在后台队列



请求路径：前端 POST /generate → 控制器仅做校验与入队并立即 202 Accepted → Worker 出队执行 → 前端轮询 /generate/status 拿阶段、日志与耗时。

两个模块共库共前端，约 3,000 行自有代码

同一进程内并存两套能力，前端顶部标签切换；共用 MySQL 库 SmartLabOS-Presales-AI。

主数据维护（移植自 DataMaintenance）

逐字保留原 UI 与操作，作为 AI 生成的数据底座。

SmartLabRepository.cs	196 行	5 类主数据 CRUD 仓储
EntityRegistry.cs	122 行	5 实体 / 列元数据
CardTemplateImporter.cs	263 行	Excel 卡片模板导入
XlsxReader.cs	149 行	零依赖 xlsx 解析
YamlEmitter / YamlMdCard.cs	168 行	导出 YAML-MD 卡片
RecordsController / TransferController	292 行	CRUD 与导入导出

售前方案自动生成（AI Agent • 新建）

把「CLI 子进程 + 扫盘回读」换成官方 SDK 直调。

ClaudeAgentService.cs	485 行	★ 大模型核心
FileTools.cs	140 行	★ 受控读写边界
DocxExporter.cs	278 行	docx 导出双路径
ModuleCatalog.cs	130 行	ID 校验反幻觉
GenerationQueue / Worker / JobModels	219 行	队列与后台消费
PresalesController / Repository	623 行	REST 与持久化

外部依赖仅 6 个 NuGet 包；前端无构建链。整体是「可读优先」的单体，新人一天可通读全部核心路径。

五步 workflow: AI 负责推理, 人负责闸门

每一步都有服务端硬校验, 未达前置条件直接 400 拒绝——AI 不能跳步, 人工确认是不可绕过的关卡。



为什么把「模块确认」设成硬闸门?

模块集合是后续平台选型、工作站组装、耗时测算的共同输入。一旦选错, 两阶段生成的全部产出都会连锁失效, 且每次返工都是真金白银的 token 成本。因此系统把「最容易出错、最难回滚」的一步交还给人: AI 负责给推荐, 工程师负责做决策。未确认前, 生成接口一律 400 拒绝——这不是流程繁琐, 是把人放在最该在的位置上。

结构化输出 + 目录校验：两道防线堵死模块幻觉

做法：让模型只能吐合法 JSON

不再让模型「写 JSON 文件再回读」，而是用 `OutputConfig.Format` 传入 JSON Schema，由 API 层保证返回结构；`additionalProperties=false` 禁止自由发挥。

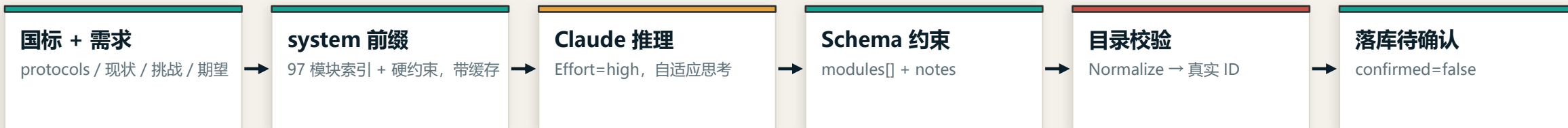
```
OutputConfig = new OutputConfig {  
    Effort = ParseEffort(_s.Effort),  
    Format = new JsonOutputFormat { Schema = schema } },  
Thinking = new ThinkingConfigAdaptive(),
```

兜底：结构合法 ≠ 内容真实

Schema 只能保证「是个字符串数组」，无法保证 MOD-CC-999 真的存在。因此返回值再过一遍 `ModuleCatalog`：以文件名为准校验存在性、统一大写、去重、保序。

```
var pick = SafeParse<ModulePick>(json);  
// 仅保留知识库中真实存在的 ID  
var ids = _catalog.Normalize(pick?.Modules);  
// 幻觉编号在此被静默丢弃
```

端到端链路



选型原则（写进提示词的硬规则）

- ① 覆盖前处理每道工序（加液 / 涡旋 / 离心 / 浓缩 / SPE 净化 / 定容）不遗漏；
- ② 满足工艺前提下尽量精简、去冗余，追求「最小充分集合」；
- ③ 确无匹配模块时写进 notes 说明缺口，绝不编造编号——「说不知道」优先于「猜一个」。

两阶段流水：先出解决方案，再整合成设计方案

两阶段各自独立发起 Agent 工具循环，共用同一份带缓存的知识库 system 前缀；阶段号由服务端权威写入 job.Phase，前端据此显示进度条。

1 阶段一：解决方案 HTML + 模块 URS

- 逐个国标处理 对每个国标 read_kb_file 读取前处理流程全文
- 强制选型范围 只能从「已确认模块」中选，清单外模块须书面说明缺口
- 平台与工作站 选定平台型号 → 搭载模组组成工作站 → 按工序顺序串联
- 耗时测算 模块功能码耗时 + 上下料时间；各平台节拍尽量对齐 600 秒，据此拆分平台
- 缺口补位 模块无法覆盖时生成「xxx-模块-URS.html」，耗时按规范值计入
- 产出 write_output 保存为 1.html、2.html...（按国标顺序）

2 阶段二：《详细设计方案》Markdown

- 数据来源 read_kb_file 读回阶段一自己写出的 *.html，必要时再读卡片
- 章节要求 封面 / 目录 / 概述 / 标准适配 / 总体架构 / 各平台 / 流程与产能 / 实施 / 风险
- 量化红线 指标必须是「数值 + 单位 + 精度 + 通讯协议」，禁止「高精度 / 快速」等定性词
- 一致性 耗时与产能必须与阶段一 HTML 的计算结果一致
- 表格化 关键技术指标表 / 检测标准适配表 / 实施阶段表
- 产出 <项目名>-详细设计方案.md → 自动导出 .docx

设计洞察：为什么阶段二要「读回自己写的 HTML」，而不是直接在内存里传？

因为落盘的 HTML 是「人可复核、可修改」的中间产物。工程师能在两阶段之间打开文件核对耗时与选型，改完再跑阶段二——中间产物即审核点。代价是阶段二要重新把 HTML 读进上下文，多花一份输入 token；这是用成本换可控性的自觉取舍。

手写 Agent 循环：一次 API 调用换一轮工具执行

RunToolLoopAsync 是整个系统的核心——不用任何 Agent 框架，用一个 for 循环把「模型请求工具 → C# 执行 → 结果回喂」串起来，最多 40 轮。



三个关键实现细节

thinking 必须原样回传

同模型多轮对话中，thinking 块与其 Signature 不可篡改，否则后续请求被拒；代码用 ThinkingBlockParam 原样带回，并单独处理 RedactedThinking。

assistant 消息完整重建

text / thinking / tool_use 全部按原顺序塞回 messages，保证模型看到的历史与它自己生成的一致。

工具结果以 user 角色追加

ToolResultBlockParam 携带 ToolUseID 配对，多个工具调用可在一轮内并行返回。

异常分层与可观测性

```
catch (AnthropicRateLimitException ex)
    → [限流] (SDK 已自动退避重试)
catch (Anthropic5xxException ex)
    → [服务端错误]
catch (AnthropicApiException ex)
    → [错误] 通用兜底
```

旧实现只能字符串匹配 CLI 的 stdout 猜错误类型；现在是强类型异常，限流由 SDK 自动退避重试。每轮还打印 usage: in / out / cacheRead / cacheWrite, token 成本实时可见。

三个受控工具：模型触碰磁盘的唯一通道

这是本项目相对旧实现最重要的改造。旧版把整个项目根目录以 `bypassPermissions` 交给 CLI；现在所有读写都由 `FileTools` 的 C# 代码亲自执行。

list_kb_dir

只读

限 references/ · potocol/ · 本项目输出目录；最多返回 400 条

read_kb_file

只读

同上三个根；单文件上限 512KB，超限只返回前 256KB

write_output

写入

仅本项目输出目录；仅 .html / .md；仅纯文件名

写入路径的四道校验 (WriteOutput)

① 非空校验

文件名为空直接拒绝

② 纯文件名

含 / \ 或非法字符即拒，杜绝目录穿越

③ 扩展名白名单

只允许 .html / .htm / .md

④ 落地路径复核

GetFullPath 后再确认未越出项目目录

旧实现 (DataMaintenance)

```
claude -p --permission-mode bypassPermissions
--add-dir <项目根>
```

模型可读写整个项目根：知识库、源码、其它客户的提案目录，无一幸免。

本项目 (受控工具)

```
_readRoots = [ ReferencesDir, ProtocolDir, projectDir ]
写入白名单根 = 单个项目输出目录
```

每个生成任务实例化一个 `FileTools` 与该项目目录绑定；越界读取一律返回不可访问，越界写入直接拒绝。

稳定前缀进缓存，易变任务进 user

system 前缀在两个阶段、每一轮工具循环中完全一致——这正是提示词缓存生效的前提。

system 前缀（带 CacheControlEphemeral）

角色与语言	SmartLabOS 售前 Agent；中文；QuEChERS / SPE / ICP-MS 等术语保留原文
三条硬约束	参数须溯源；价格转销售；只能用工具读写且限 .html/.md
知识库导航	01-modules / 02-platforms / 03-workstation / 04-solutions / 06-pallet / potocol/MD
97 模块全清单	「ID — 名称（分类）」仅作索引，参数须 read_kb_file 读卡片
提案 HTML 模版	整份模版文件内联注入，进程内缓存一次读取

user 消息（每阶段各异）

三个 Build*Prompt 方法按项目动态拼装：已确认模块清单（强制选型范围）、国标列表、客户需求（现状/挑战/期望/流程范围/上下料/软件功能），再叠加该阶段的具体要求。

```
BuildModuleSelectionPrompt(p)  → 模块选定
BuildHtmlPhasePrompt(p)       → 阶段一
BuildDetailPhasePrompt(p)     → 阶段二
BuildPreview(p)               → 前端「指令预览」复用阶段一
```

上下料方式还会切换措辞：自动 / 人工 / 未指定（按卡片默认值），让耗时估算口径明确。

模块清单注入 system 而非 user，是刻意的：它随知识库变化但在一次任务内恒定，放前缀才能吃到缓存。

成本模型：为什么「按需读取」比「全盘塞入」更省

旧实现每次把整个知识库读进上下文；本项目只在 system 放「索引 + 导航 + 模版」，具体卡片参数由模型用 read_kb_file 按需拉取——一次任务通常只读十余个真正相关的模块卡片。稳定前缀命中缓存后按 cacheRead 计价，日志每轮打印 in / out / cacheRead / cacheWrite 四项用量，成本可核算。
注意：CacheControlEphemeral 默认 5 分钟 TTL——阶段一若耗时较长，阶段二起手可能已经错过缓存窗口。

长任务不占 HTTP：入队即返回，轮询看进度

方案生成动辄十几分钟到一小时，绝不能压在一个 HTTP 请求里。控制器只做校验与入队，立刻 202 Accepted。

1	控制器	校验前置条件 → 建 Job → 入队 → 202 Accepted	IsGenerating(id) 判重，同项目不允许并发
2	有界队列	Channel.CreateBounded(200)	FullMode=Wait 产生背压；SingleReader 单消费者
3	工作者	BackgroundService + SemaphoreSlim(2)	先拿信号量再 Task.Run，并发上限保护 API 限额
4	超时	CreateLinkedTokenSource + CancelAfter(60min)	与进程停止令牌联动，可优雅取消
5	双写状态	JobRegistry (内存，供轮询) + MySQL (供留痕)	阶段号 / 日志 / 耗时 / 产物清单 / 结束码

服务端权威进度

阶段号由 ClaudeAgentService 通过 setPhase(1|2) 回调写入，而非前端猜测。status 接口返回 phase / phaseCount / phaseLabel / elapsedSeconds / 日志尾部 6000 字符。

但队列仍是「进程内」的

Channel 与 JobRegistry 都在内存：进程重启后排队任务凭空消失，运行中任务在库里永远停在 running。README 称「进程退出时不再凭空丢任务」——这一点尚未真正兑现。

两张表 + 一个目录，就是全部状态

SmartLabOS-PresalesProject

项目需求主表，project_name 唯一且同时作为 projects/ 下的目录名。

protocols / challenges / expectations / process_scope	JSON 数组列，由仓储负责序列化
modules / modules_confirmed	已选定模块 ID 数组 + 人工确认标志
gen_status	draft / queued / running / succeeded / failed
last_command_file / last_generated_at	最近一次生成的标记与时间

SmartLabOS-PresalesGeneration

每次生成一条执行记录，服务重启后仍可回溯历史。

command_file	本次生成标记 Anthropic-Messages-API- <时间戳>
status / exit_code	queued / running / succeeded / failed
output_files	JSON 数组：产物文件名清单
log_excerpt	LONGTEXT，运行日志尾部 20000 字符

交付产物：projects/<项目名>/

解决方案

1.html · 2.html ...

每个国标一份，含 SVG / Mermaid 图示与耗时测算

需求说明书

xxx-模块-URS.html

已确认模块无法覆盖时，为新建模块生成 URS

详细设计

<项目名>-详细设计方案.md

整合全部 HTML，章节完整、指标可量化

可交付

<项目名>-详细设计方案.docx

自动导出，客户可直接接收的 WORD

前端「生成的文件」列表通过 GET /files 枚举 *.html / *.md / *.docx，GET /file?name= 按扩展名回不同 Content-Type，可在线预览或下载。

docx 双路径：优先保真，永远有兜底

交付物必须是 WORD。系统不把这件事交给模型，而是对「我们已生成并可复核的 Markdown」做确定性转换。

1 Pandoc (首选)

保真度最高。若本机装了 pandoc 或 Agent:PandocPath 指向可执行文件即启用。进程带 120 秒超时与强杀保护；未安装时捕获 Win32Exception 静默回退，不记为错误。

```
pandoc <md> --from gfm  
-o <docx>
```

2 内置 OpenXML (兜底)

零外部依赖、离线可用。用 OpenXml 在进程内把 Markdown 转 docx：标题 / 段落 / 列表 / GFM 表格 / 代码块 / 粗斜体。不耗任何额外 API 成本。

```
GET /api/dev/docx-selftest  
→ {"ok":true,"engine":"openxml"}
```

3 Agent Skills (未启用)

可让 Claude 用 code_execution + 官方 docx 技能在托管容器内直接渲染。刻意不采用：额外 API 成本、依赖 beta 特性，且是「重新生成」而非转换已复核的 Markdown。

```
Container.Skills =  
[{"skill_id": "docx"}]
```

这个取舍值得单独说明

「让模型直接生成 docx」听起来更 AI，但它把一个确定性问题重新变回概率问题。

Markdown 已经是经过人工复核的中间产物——从它到 docx 是纯格式转换，没有任何需要「智能」的地方。交给模型意味着：额外 token 成本、beta 特性依赖、每次结果可能不同、且无法保证与已复核内容一致。

系统的选择是：AI 只做需要判断力的事（选型、推理、书写），确定性的事交给确定性的代码。这条边界划得很清楚，也是整个项目最成熟的设计观。

从「驱动一个 CLI」到「构建一个 Agent」

同一业务，两代实现。差别不在功能清单，而在「谁掌握控制权」——旧版把控制权交给子进程，新版把控制权收回进程内。

	旧实现 • DataMaintenance	本项目 • Presales AI Agent
大模型调用	● Process.Start(claude -p ...) 子进程	● 官方 SDK client.Messages.Create, 进程内 HTTPS
权限模型	● bypassPermissions + --add-dir 项目根 (越权)	● 3 个受控工具: 读限知识库、写限本项目 + 扩展名白名单
结构化结果	● 让模型写 JSON 文件, 再扫盘回读	● OutputConfig.Format 传 JSON Schema, API 层保证结构
知识库成本	● 每次全盘读入上下文	● system 放索引, 卡片按需 read_kb_file; 前缀走提示词缓存
错误处理	● 字符串匹配 stdout 猜错误	● 强类型异常分层 + SDK 自动退避重试 + usage 用量可读
后台任务	● Task.Run 即发即忘, 纯内存态	● 有界 Channel + BackgroundService + 并发上限 + 状态落库
机密管理	● 明文口令入仓	● user-secrets / 环境变量注入, 仓库零机密
WORD 交付	● 依赖 CLI 的 docx 技能 (不确定)	● 内置 Pandoc / OpenXML 双路径, 离线、零额外成本

本机实测: 构建 0 警告 0 错误 (.NET 10.0.301 / C# 14) ; /api/health 数据库 connected; options 返回 11 个国标; modules 返回 97 个模块; docx 自检 engine=openxml。

这个项目做对了什么

1 安全边界是设计出来的，不是补上去的

FileTools 不是一个「加了校验的工具类」，而是从架构上确立了「模型不直接碰磁盘」的原则。读根白名单、写扩展名白名单、纯文件名强制、路径复核四道防线层层收敛，把旧版 bypassPermissions 的越权面从「整个项目根」压缩到「单个项目输出目录」。

3 人机边界划得清楚

AI 做推理（选型、组装、书写），人做决策（模块确认），代码做确定性的事（docx 转换、ID 校验、参数化 SQL）。「模块确认」硬闸门与「拒绝用模型渲染 docx」是同一种判断力的两次体现。

5 成本意识贯穿实现

system 前缀带缓存、知识库按需读取而非全量塞入、每轮打印 in/out/cacheRead/cacheWrite 四项用量、模版进程内缓存一次读取——成本不是事后优化，是写在第一版里的。

2 反幻觉是双层的：结构 + 内容

Schema 保证结构合法，ModuleCatalog 保证内容真实。以文件名为模块 ID 这一决定尤其巧妙——绕开了 YAML 解析的脆弱性，让「ID 存在性」变成一次文件系统查询，可靠且零歧义。

4 中间产物即审核点

阶段一的 HTML 落盘后可被人工打开、核对、修改，阶段二再读回整合。工程师被放在流程内部而非末端——这是提案类系统能否真正落地的关键。代价是阶段二要重读 HTML、多花一份输入 token，属于用成本换可控性的自觉取舍。

6 工程基本功扎实

机密零入仓、Swagger 仅开发环境、全参数化查询、有界队列产生背压、任务级超时与令牌联动、UTF-8 BOM 兼顾记事本、静态文件禁缓存——每一处都是踩过坑的人才会写的代码。

一次关于「AI 该做什么」的 克制而清醒的实践

架构上

五层单体，请求即返回、重活进后台队列。没有微服务、没有前端构建链、没有 Agent 框架——用最少的抽象把事情做成，新人一天可通读全部核心路径。

方法上

AI 只做需要判断力的事：选型推理、方案书写。结构校验、ID 存在性、docx 转换这些确定性问题，一律交给确定性代码。这条边界是整个项目最值钱的设计观。

下一步

当前形态更像一个「跑得通的强原型」：正确性与安全的关键洞（鉴权、截断静默）需要优先补上，之后才谈得上流式、质检与规模化。